Le Timer0 et les interruptions du PIC16F84A

Polo Antoine

1. INTRODUCTION

Dans l'introduction au PIC16F84A, nous avons fait clignoter une LED à l'aide de la fonction bloquante `__delay_ms()`. Cette approche est simple, mais le microcontrôleur ne peut rien faire d'autre pendant qu'il attend.

Dans ce document, nous allons utiliser le Timer0, le timer 8 bits intégré au PIC16F84A, ainsi que ses interruptions, afin de faire clignoter deux LEDs à des fréquences différentes, sans jamais bloquer le programme. Nous détaillerons pour cela les registres `OPTION_REG`, `TMR0` et `INTCON`.

2. LE TIMER0

Le PIC16F84A possède un timer de 8 bits. Par défaut, il est incrémenté par l'horloge interne du système, qui divise par 4 la fréquence du quartz. Avec une fréquence de 4 MHz, il est donc incrémenté toutes les 1 µs.

Le registre `OPTION_REG` permet de générer un prédiviseur (*prescaler*) allant de 2 à 256, ce qui permet de ralentir le timer.

3. CALCULS

Prenons l'exemple d'une action à effectuer toutes les 10 ms, avec un quartz de 4 MHz.

Période d'incrémentation :

$$\frac{F_{OSC}}{4} = \frac{4\ \text{MHz}}{4} = 1\ \text{MHz} \Rightarrow \frac{1}{1\ \text{MHz}} = 1\ \mu\text{s}$$

Choix du prédiviseur (ici 64) : le temps pour un tic du timer vaut $1\ \mu\text{s} \cdot 64 = 64\ \mu\text{s}$.

Nombre de tics pour 10 ms :

$$N = \frac{\text{Temps souhaité}}{\text{Temps par tic}} = \frac{10\,000\ \mu\text{s}}{64\ \mu\text{s}} \approx 156\ \text{tics}$$

On peut donc maintenant changer l'adresse de départ de TMR0 à :

$$\text{Valeur départ} = 256 - N = 256 - 156 = 100\ (0\text{x}64)$$

4. REGISTRES IMPORTANTS

4.1. Registre de configuration (OPTION_REG)

Ce registre de 8 bits configure le Timer0 ainsi que quelques fonctions du PORTB.

Table 1. Bits du registre OPTION_REG.

b7	b6	b5	b4	b3	b2	b1	b0
nRBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0

- `nRBPU` : contrôle les résistances de pull-up du PORTB, le pull-up est activé à 0.
- `INTEDG` : sens de l'interruption externe (RB0), 1 pour front montant et 0 pour descendant.
- `T0CS` : source d'horloge du timer, 0 pour l'horloge interne et 1 pour l'horloge externe.
- `T0SE` : front d'horloge externe (si T0CS à 1), 0 pour front montant et 1 pour descendant.
- `PSA` : assignation du prédiviseur, à 0 il est assigné à Timer0 et à 1 pour le watchdog.
- `PS2-0` : détermine la valeur du prédiviseur (2, 4, 8, 16, 32, 64, 128, 256). Il utilise une valeur binaire de 000 à 111.

Lorsque le prédiviseur est assigné au Timer0 (`PSA = 0`), les bits `PS2-0` correspondent aux valeurs de la Table 2.

Table 2. Valeur du prédiviseur selon PS2-0.

PS2-0	Division	PS2-0	Division
000	1:2	100	1:32
001	1:4	101	1:64
010	1:8	110	1:128
011	1:16	111	1:256

4.2. Le registre du timer (TMR0)

`TMR0` est le compteur du timer. C'est un registre de 8 bits, lisible et modifiable, qui est incrémenté à chaque tic. Lorsqu'il passe de 255 (0xFF) à 0 (0x00), on parle de débordement (*overflow*) et le drapeau `T0IF` est levé.

Pour obtenir une durée précise, on ne laisse pas le timer partir de 0 : on le recharge avec la valeur de départ calculée à la section précédente. Il ne reste alors que 256 − départ tics avant le débordement.

4.3. Registre des interruptions (INTCON)

Le registre `INTCON` contient les bits d'activation (*enable*) et les drapeaux (*flags*) des principales sources d'interruption.

Table 3. Bits du registre INTCON.

b7	b6	b5	b4	b3	b2	b1	b0
GIE	EEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF

- `GIE` : *Global Interrupt Enable*, active (1) ou désactive (0) toutes les interruptions de façon globale.
- `EEIE` : active l'interruption de fin d'écriture dans l'EEPROM.
- `T0IE` : active l'interruption de débordement du Timer0.
- `INTE` : active l'interruption externe sur la broche RB0/INT.
- `RBIE` : active l'interruption sur changement d'état des broches RB4 à RB7.
- `T0IF` : drapeau de débordement du Timer0, mis à 1 par le matériel quand TMR0 passe de 255 à 0.
- `INTF` : drapeau de l'interruption externe RB0/INT.
- `RBIF` : drapeau de changement d'état sur RB4 à RB7.

Note

Les drapeaux ne sont jamais remis à 0 automatiquement : c'est au programme de le faire dans la routine d'interruption, sans quoi elle serait relancée en boucle.

Pour que le Timer0 déclenche une interruption, il faut donc que `T0IE` et `GIE` soient tous les deux à 1.

5. MONTAGE

Le montage reprend le circuit de base de l'introduction (quartz, condensateurs et résistance), auquel on ajoute une seconde LED :

- un quartz de 8 MHz entre les broches OSC1 et OSC2, chacune reliée à la masse par un condensateur de 22 pF ;
- une résistance de 10 kΩ entre la broche MCLR et le +5 V ;

- une première LED, avec sa résistance de limitation, sur la broche RB0 ;
- une seconde LED, avec sa résistance de limitation, sur la broche RB1 ;
- l'alimentation sur VDD et VSS (via le PICkit 5 pour la programmation).

6. EXEMPLE DE CODE

Le code ci-dessous permet de faire clignoter deux LEDs à des fréquences différentes sur RB0 et RB1 avec un quartz de 8 MHz.

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = OFF
#pragma config CP = OFF

#include <xc.h>

#define _XTAL_FREQ 8000000

volatile uint16_t counter_ms = 0;

void __interrupt() isr(void) {
    if (T0IF) { // Drapeau d'interruption de TMR0
        TMR0 = 0x06;
        T0IF = 0;
        counter_ms++;
    }
}

void main(void) {
    TRISBbits.TRISB0 = 0;
    TRISBbits.TRISB1 = 0;

    PORTBbits.RB0 = 0;
    PORTBbits.RB1 = 0;

    OPTION_REG = 0b00000010;

    TMR0 = 0x06;
    T0IE = 1; // Active l'interruption du timer
    GIE = 1; // Active les interruptions de façon globale

    uint16_t last_time_led1 = 0;
    uint16_t last_time_led2 = 0;

    while (1) {
        if ((uint16_t)(counter_ms - last_time_led1) >= 100) {
            last_time_led1 = counter_ms;
            PORTBbits.RB0 = !PORTBbits.RB0;
        }

        if ((uint16_t)(counter_ms - last_time_led2) >= 1000)
    ↪ {
            last_time_led2 = counter_ms;
            PORTBbits.RB1 = !PORTBbits.RB1;
        }
    }
}
```

Code 1. Clignotement de deux LEDs avec le Timer0.

6.1. Calcul des valeurs utilisées

Ce programme utilise un quartz de 8 MHz et cherche à générer une interruption toutes les 1 ms :

- Horloge du timer : 8 MHz/4 = 2 MHz, soit 0,5 µs par incrément.
- `OPTION_REG = 0b00000010` : `PSA = 0` (prédiviseur sur Timer0), `PS2-0 = 010` (division par 8), horloge interne (`T0CS = 0`) et pull-ups activés (`nRBPU = 0`). Un tic dure donc $0{,}5 \times 8 = 4\ \mu$s.
- Pour 1 ms : $N = 1000/4 = 250$ tics, donc une valeur de départ de $256 - 250 = 6$, soit `0x06`.

6.2. Fonctionnement

- **Initialisation** : `TRISB0` et `TRISB1` à 0 configurent RB0 et RB1 en sorties, puis les deux LEDs sont éteintes. `T0IE = 1` et `GIE = 1` autorisent l'interruption du Timer0.
- **Interruption** : toutes les millisecondes, le Timer0 déborde et lève `T0IF`. La routine `isr` recharge `TMR0`, remet `T0IF` à 0 et incrémente `counter_ms`. Cette variable est déclarée `volatile` car elle est modifiée dans l'interruption et lue dans le programme principal.
- **Boucle principale** : on compare `counter_ms` à l'instant du dernier changement de chaque LED. La LED de RB0 change d'état toutes les 100 ms et celle de RB1 toutes les 1000 ms. La soustraction en `uint16_t` reste correcte même lorsque `counter_ms` repasse à 0.

Contrairement à `__delay_ms()`, rien ne bloque le programme : on peut ajouter autant de tâches temporelles que nécessaire dans la boucle.

7. CONCLUSION

Le Timer0 et ses interruptions permettent de mesurer le temps sans bloquer le microcontrôleur. Il suffit de calculer la valeur de départ de `TMR0` à partir de la fréquence du quartz et du prédiviseur choisi dans `OPTION_REG`, puis d'activer l'interruption avec `INTCON`.

À partir d'un compteur en millisecondes, on peut gérer plusieurs actions périodiques indépendantes, comme ici le clignotement de deux LEDs à des fréquences différentes.